Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval)

A Practical Evaluation Procedure for Neural Network Optimization Algorithms

Silver Moon (`cloverxion`)

Lumina Moon (`cloverx-id`)

November 2026

Technical Specification — Version 1.1.1

Abstract

This document describes the **Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.1)**, a general testing and benchmarking procedure for neural network optimizers and their execution backends. It defines a four-stage evaluation pipeline for testing first-order, state-compressed, and preconditioned optimization algorithms across CPU and GPU implementations before deployment. Rather than relying on ad-hoc training runs, LSO-Eval organizes evaluation from cheap mathematical checks to full-scale model training: (I) Algebraic Safety and Numerical Precision Checks, (II) Synthetic Test Surfaces, Cross-Backend Consistency, and Checkpoint Roundtrip Verification, (III) Short-Run Proxy Model Screening, and (IV) Large-Scale Pre-Training Evaluation with Train–Validation Deduplication and Two-Class Acceptance Criteria. In addition, **V-Eval** provides a dedicated evaluation track for computer vision architectures across low-, medium-, and full-resolution datasets, and **D-Eval** defines regression checks across software versions. Reference case boxes throughout this document illustrate how the procedure is applied in practice using the **LuminaV** optimizer (`v1.5.3`).

---

1 Scope, Principles, and the Staged Evaluation Pipeline

1.1 Objective and Operational Scope

LSO-Eval provides a shared evaluation checklist for testing optimization algorithms and hardware execution kernels within Lumina Moon projects. Because LSO-Eval is designed as an algorithm-agnostic procedure, its version number (`v1.1.1`) is independent of any specific optimizer release. The procedure focuses on four practical goals:

1. **Save Compute Resources:** Catch algebraic errors, numerical overflow/underflow, or narrow hyperparameter ranges on CPU before running expensive GPU experiments.
2. **Separate Mathematical Issues from Implementation Bugs:** Test update formulas, function approximations, low-precision rounding, and hardware kernels in separate steps so failures are easy to diagnose.
3. **Verify Cross-Backend Consistency:** Ensure that vectorized CPU kernels, GPU shaders/kernels, and Python bindings produce consistent numerical trajectories relative to a simple scalar reference implementation.

4. **Compare Quality and Memory Fairly:** Evaluate candidate optimizers on both validation quality and isolated optimizer memory usage (separating optimizer state memory from fixed model weights and activations), with clear criteria for full-state (two-buffer) and compressed-state (single-buffer) designs.

1.2 Staged Filtering Protocol

The LSO-Eval workflow is organized into two operational tiers across four core stages plus vision and regression extensions (Table 1). **Stages I and II** run on CPU in seconds and serve as the routine continuous integration (CI) check for code changes. **Stages III, IV, and V-Eval** run on GPU hardware to evaluate empirical convergence and hardware footprint when introducing or revising an optimization algorithm. If a candidate fails an earlier stage, testing stops until the underlying issue is fixed.

Table 1: The LSO-Eval Staged Evaluation Pipeline (v1.1.1)

Stage	Designation	Core Verification Objective	Typical Cost
Stage I	Math & Precision	No division-by-zero, bounded update velocity, half-precision floor safety, function approximation accuracy, and unbiased low-precision rounding.	$\mathcal{O}$(ms)
Stage II	Synthetic & Parity	Ill-conditioned traversal ($\kappa \geq 1000$), saddle escape, hyperparameter stability, single-pass fusion parity, cross-backend consistency, and checkpoint roundtrip.	$\mathcal{O}$(sec)
Stage III	Short-Run Proxy	Layer-wise update-to-weight ratio bounds ($R_{t,l} \in [10^{-7}, 0.10]$), loss spike recovery ($F_2$), and BF16/FP16 rounding underflow check ($F_3 < 0.25$).	$\mathcal{O}$(min)
Stage IV	Large-Scale Run	Train–validation deduplication, low-precision BF16/FP16 stability, isolated optimizer VRAM (VRAM_{opt}), step latency, and Two-Class held-out perplexity checks.	$\mathcal{O}$(hours)
V-Eval	Vision Benchmarks	Track V-A (from-scratch training across CIFAR-100, ImageWoof 224^2, Mini-ImageNet 128^2, and CIFAR-10) and Track V-B (downstream fine-tuning) across 6 contenders.	$\mathcal{O}$(hours)
D-Eval	Version Regression	4-gate comparison checking a new release candidate v_k against the previous version v_{k-1}.	Cross-Stage

2 Stage I: Algebraic Safety & Numerical Precision Checks

Stage I checks the mathematical properties and low-precision numerical behavior of an optimizer before running any neural network training loop. The update equations are expressed in symbolic arithmetic and tested with an SMT solver and statistical checks.

2.1 General SMT Safety Checklist (Z3 Solver)

To verify that an update rule $u_t(\theta, g, m, v)$ cannot produce NaNs or unbounded jumps under valid hyperparameters, the candidate is checked against seven general safety conditions using the Z3 Theorem Prover [1]. Each condition is verified by showing that its **negation** is `UNSAT` (unsatisfiable):

Q1. Denominator Non-Singularity (No Division-by-Zero): For any gradient $g \in \mathbb{R}$, momentum state $m \in \mathbb{R}$, and positive epsilon $\epsilon > 0$, prove that the update denominator $\mathcal{E}$ (or variance divisor σ) satisfies $\mathcal{E} > 0$. Negated query: $\mathcal{E} \leq 0$.

Q2. Update Magnitude & Trust-Region Bounds: Verify that the update velocity and any optional trust-region clipping match the optimizer's design profile:

- *Profile A (Normalized / Bounded-Velocity Optimizers, e.g., Lion [3], LuminaV):* Bounded update magnitude $|u_t| \leq 1.0$, positive coordinate clipping bound $B_i > 0$, and radial clipping lower bound $R \geq r_{\text{bound}}$ [4] when adaptive clipping is enabled.
- *Profile B (Variance-Adaptive Optimizers, e.g., AdamW [5], AdaBelief [6], ADOPT [7]):* Clamped second-moment divisor $\sigma_{\text{clamped}} \geq \epsilon_{\min} > 0$ and finite update magnitude as $g \to 0$.
- *Profile C (Matrix-Preconditioned / Subspace Optimizers, e.g., Shampoo [8], GaLore [9]):* Bounded operator norm $\|U_t\|_2 \leq C$ and non-singular preconditioner $\sigma_{\min}(P) > 0$.

Q3. Radial Scaling & Alignment Mask Bounds: If the optimizer uses radial norm scaling $S(r)$ for $r \geq 0$, prove that $0 < S(r) \leq 1.0$. If the optimizer uses cautious gradient-alignment masking [10] with density floor $c_{\min} \in (0, 1]$, prove that the rescaling factor $\gamma_{\text{cautious}} = 1/\text{clamp}(\bar{m}, c_{\min}, 1.0)$ lies in $[1.0, 1/c_{\min}]$.

Q4. Half-Precision Floor & Overflow Bounds: For 16-bit execution (`float16` or `bfloat16`), verify that any lower clipping floor remains above the IEEE 754 FP16 subnormal threshold ($\text{floor}_{\text{sub}} = 6.1035 \times 10^{-5}$) and any FP16 variance accumulator is clamped below the FP16 overflow limit (65,504.0).

Q5. Descent Alignment: Prove that from zero initial momentum ($m_0 = 0$) or under cautious alignment masking [10] ($m_{\text{mask}} = \mathbb{I}(u_t \cdot g_t > 0)$), the effective update aligns non-negatively with the current gradient: $u_0 \cdot g_t \geq 0$ and $\delta_{\text{cautious}} \cdot g_t \geq 0$.

Q6. Weight Decay Contraction: For learning rate $\eta \in (0, 0.05]$ and decoupled weight decay [5] $\lambda_{\text{wd}} \in [0, 0.2]$, prove that the decay multiplier satisfies $(1 - \eta\lambda_{\text{wd}}) \in (0, 1]$.

Q7. Bias Correction Positivity: For all steps $t \geq 1$ and $\beta_1, \beta_2 \in (0, 1)$, prove that any first- or second-moment bias-correction factors remain positive ($\text{bc}_1 > 0, \text{bc}_2 > 0$).

2.2 Symbolic Limits, Approximation Error & Low-Precision Rounding Checks

Alongside SMT verification, Stage I includes four analytical and numerical precision checks:

S1. Large-Gradient Limit Check: Using SymPy [12], evaluate the symbolic limit of the

update velocity u_t as gradient magnitude grows large:

$$u_\infty = \lim_{|g|\to\infty} u_t(m, g). \tag{1}$$

Acceptance Rule: For Profile A, the limit must satisfy $|u_\infty| \leq 1.0$. For Profiles B and C, the limit must remain finite without overflow.

S2. Piecewise Transition Continuity (C^0/C^1): If an update rule switches between piecewise branches at a small threshold r_{thresh} (for example, replacing $\tanh(r)/r$ with 1.0 near zero), verify that the transition jump Δ_{jump} is smaller than single-precision machine epsilon ($\epsilon_{\text{FP32}} \approx 1.19 \times 10^{-7}$).

S3. Custom Transcendental Approximation Accuracy (When Applicable): If an implementation replaces standard library functions (such as tanh, erf, or $1/\sqrt{x}$) with rational or polynomial approximations [13] for SIMD/GPU execution, measure the maximum absolute error across the operating domain against IEEE 754 `float32`. **Acceptance Rule:** Maximum absolute error must satisfy $\|f_{\text{approx}}(x) - f_{\text{exact}}(x)\|_\infty < 1.5 \times 10^{-6}$, while preserving odd/even symmetry and NaN propagation.

S4. Stochastic Rounding Unbiasedness (When Applicable): If the optimizer quantizes states or parameters into 16-bit/8-bit formats using stochastic rounding [14, 15], draw $N = 200{,}000$ samples across both normal ($[10^{-3}, 10^{-1}]$) and FP16 subnormal ($[10^{-5}, 6.1 \times 10^{-5}]$) ranges. Test unbiasedness ($H_0 : E[\mathcal{Q}_{\text{SR}}(x)] = x$) using a two-sided Student's t-test and Two One-Sided Tests (TOST) within tolerance $\pm\delta_{\text{tol}} = 10^{-6}$. **Acceptance Rule:** Sample mean bias must satisfy $|\mu_{\mathcal{Q}} - x| < 10^{-6}$ ($p_{\text{TOST}} < 0.01$, or t-test $p \geq 0.01$).

Reference Case Example: LuminaV (v1.5.3) Stage I Verification (Z3 SMT, Padé [13/6], and MurmurHash3 SR)

In LuminaV (`v1.5.3`), the first moment updates via $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$. Single-Buffer TD-CLM (`buffer=1`) and Dual-Buffer Convex Lookahead Momentum (`buffer=2`) compute update velocity using lookahead interpolation [11], triangle-defect norm envelopes [2], and belief variance [6, 7]:

$$u_t^{(\text{B1})} = \frac{w_1 m_{t-1} + w_2 g_t}{w_1 |m_{t-1}| + w_2 |g_t| + \lambda |d_t^{(\text{B1})}| + \epsilon_{\text{eff}}}, \quad w_1 = \beta_1^2, \quad w_2 = 1 - \beta_1^2,$$

$$u_t^{(\text{B2})} = \tanh\left(\frac{\beta_1 m_t + (1-\beta_1) g_t}{\max(c_1 \sqrt{\max(v_t, 0)} + c_2, 10^{-12})} \right), \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) \left(d_t^{(\text{B2})} \right)^2,$$

where $d_t^{(\text{B1})} = g_t - m_{t-1}$, $d_t^{(\text{B2})} = g_t - m_t$, $c_1 = \frac{(1-\beta_1^{t+1})\tau}{\sqrt{\max(1-\beta_2^t, 10^{-15})}}$, and $c_2 = \epsilon_{\text{eff}}(1-\beta_1^{t+1})\tau$. Note the identity $\beta_1 m_t + (1-\beta_1) g_t \equiv \beta_1^2 m_{t-1} + (1-\beta_1^2) g_t = w_1 m_{t-1} + w_2 g_t$.

LuminaV Implementation Details Tested in Stage I (`eval/modules/smt.py` & `math_verification.py`):

- *Padé* [13/6] *Rational* tanh*:* For AVX-512/AVX2 and Vulkan SPIR-V backends, $\tanh(x)$ is evaluated via a clamped [13/6] rational Padé approximant [13] on $x \in [-7.9988117, 7.9988117]$, achieving a maximum error of $\mathbf{9.48 \times 10^{-8}} < 1.5 \times 10^{-6}$ across 500,000 points.

- *MurmurHash3 Bitwise Stochastic Rounding:* Low-precision BF16/FP16 rounding uses a stateless 32-bit MurmurHash3 finalizer seeded per parameter and step ($\text{seed}_m = (\text{seed}_0 + t \cdot \texttt{0x9E3779B9} + \text{idx}_{\text{param}} \cdot 10007)$ & $\texttt{0x7FFFFFFF}$), achieving a maximum empirical bias of $\mathbf{3.53 \times 10^{-7}} < 10^{-6}$ ($p_{\text{TOST}} = 0.00 < 0.01$) across 200,000 normal and FP16 subnormal samples.
- *SymPy Limit & Equilibrium:* For Buffer 1 ($\beta_1 = 0.9, \lambda = 0.3$), $u_\infty = \frac{w_2}{w_2+\lambda} = \frac{0.19}{0.49} \approx 0.3878 < 1.0$ and $\lambda^* = \frac{2(0.19)^2}{0.24} \approx 0.3008$. At $r_{\text{thresh}} = 10^{-4}$, the radial continuity jump is $\Delta_{\text{jump}} \approx 3.33 \times 10^{-9} \ll \epsilon_{\text{FP32}}$.

Listing 1: LuminaV Reference Stage I.1: 12-Query Z3 SMT Verification Script (eval/modules/smt.py)

```python
from z3 import If, Or, Real, Reals, Solver, unsat

def run_z3_smt_audit() -> bool:
    solver = Solver()
    m, g, w1, w2, lam, eps = Reals("m g w1 w2 lam eps")
    base_conds = [w1 > 0, w2 > 0, (w1 + w2) == 1, lam >= 0, eps > 0]
    abs_m, abs_g = If(m >= 0, m, -m), If(g >= 0, g, -g)
    diff = g - m
    abs_diff = If(diff >= 0, diff, -diff)
    clm_m = w1 * m + w2 * g
    clm_env = w1 * abs_m + w2 * abs_g + lam * abs_diff + eps
    abs_clm_m = If(clm_m >= 0, clm_m, -clm_m)

    # 1-2. TD-CLM Non-Singularity (Q1) & Unit Metric Bound |u| < 1.0 (Q2)
    solver.push(); solver.add(*base_conds, clm_env <= 0)
    res_tdclm_div0 = solver.check(); solver.pop()
    solver.push(); solver.add(*base_conds, abs_clm_m >= clm_env)
    res_tdclm_bound = solver.check(); solver.pop()

    # 3-4. Buffer-2 Sigma Positivity (Q1/Q2) & Cautious Scale [1, 1/c_min] (Q3)
    sqrt_v, c1, c2, m_bar, c_min = Reals("sqrt_v c1 c2 m_bar c_min")
    sigma = sqrt_v * c1 + c2
    sigma_clamped = If(sigma >= 1e-12, sigma, 1e-12)
    solver.push(); solver.add(sqrt_v >= 0, c1 > 0, c2 > 0, Or(sigma <= 0,
    sigma_clamped <= 0))
    res_sigma = solver.check(); solver.pop()
    solver.push(); solver.add(c_min == 0.5, m_bar >= c_min, m_bar <= 1.0)
    solver.add(Or((1.0 / m_bar) < 1.0, (1.0 / m_bar) > (1.0 / c_min)))
    res_cautious = solver.check(); solver.pop()

    # 5-6. Coordinate Clipping Bound B > 0 & Radial Clipping Lower Bound R >=
    B_ratio (Q2)
    p, B_ratio, norm_p = Reals("p B_ratio norm_p")
    solver.push(); solver.add(B_ratio > 0, (B_ratio * B_ratio) * (p * p + 1e-4) <= 0)
    res_coord_b = solver.check(); solver.pop()
    solver.push(); solver.add(B_ratio > 0, norm_p >= 0)
    R = If(norm_p >= 1.0, B_ratio * norm_p, B_ratio * 1.0)
    solver.add(R < B_ratio); res_radial_r = solver.check(); solver.pop()

    # 7-8. FP16 Safe-Zone Floor >= 1.5e-4 & Subnormal/Variance Safety (Q4)
    p_fp16, v_raw = Reals("p_fp16 v_raw")
    p_scaled = 0.01 * If(p_fp16 >= 0, p_fp16, -p_fp16)
    b_safe = If(p_scaled >= 1.5e-4, p_scaled, 1.5e-4)
    v_fp16 = If(v_raw <= 65000.0, v_raw, 65000.0)
    solver.push(); solver.add(b_safe < 1.5e-4); res_fp16 = solver.check();
    solver.pop()
    solver.push(); solver.add(Or(b_safe <= 6.1035e-5, v_fp16 > 65504.0))
    res_subnormal = solver.check(); solver.pop()

```

```
    # 9-12. Radial Scale (Q3), Bias Corr (Q7), Weight Decay (Q6), Descent (Q5)
    r, lr, wd = Reals("r lr wd")
    solver.push(); solver.add(r >= 0)
    s_radial = If(r == 0, 1.0, (15.0 + r * r) / (15.0 + 6.0 * r * r))
    solver.add(Or(s_radial <= 0.0, s_radial > 1.0)); res_radial_scale =
    solver.check(); solver.pop()

    beta1, beta2, b1_tp1, b2_t = Reals("beta1 beta2 b1_tp1 b2_t")
    solver.push(); solver.add(beta1 > 0, beta1 < 1, beta2 > 0, beta2 < 1,
               b1_tp1 > 0, b1_tp1 <= beta1 * beta1, b2_t > 0, b2_t <= beta2)
    bc2_clamped = If(1.0 - b2_t >= 1e-15, 1.0 - b2_t, 1e-15)
    solver.add(Or((1.0 - b1_tp1) <= 0, bc2_clamped <= 0)); res_bias_corr =
    solver.check(); solver.pop()

    solver.push(); solver.add(lr > 0, lr <= 0.05, wd >= 0, wd <= 0.2)
    solver.add(Or((1.0 - lr * wd) <= 0, (1.0 - lr * wd) > 1)); res_wd =
    solver.check(); solver.pop()

    g_val, u_val, w2_val, lam_val, eps_val, gamma_c = Reals("g_val u_val w2_val
    lam_val eps_val gamma_c")
    solver.push(); solver.add(w2_val > 0, lam_val >= 0, eps_val > 0, gamma_c >= 1.0)
    abs_g_val = If(g_val >= 0, g_val, -g_val)
    u_zero_m = (w2_val * g_val) / (w2_val * abs_g_val + lam_val * abs_g_val +
    eps_val)
    delta_cautious = u_val * If((u_val * g_val) > 0, 1.0, 0.0) * gamma_c
    solver.add(Or(u_zero_m * g_val < 0, delta_cautious * g_val < 0)); res_descent =
    solver.check(); solver.pop()

    return all(s == unsat for s in [
        res_tdclm_div0, res_tdclm_bound, res_sigma, res_cautious, res_coord_b,
    res_radial_r,
        res_fp16, res_subnormal, res_radial_scale, res_bias_corr, res_wd, res_descent
    ])
```

3 Stage II: Synthetic Test Surfaces, Cross-Backend Parity & State Serialization

Stage II tests optimizer trajectories on classical optimization surfaces, checks sensitivity to hyperparameter changes, verifies numerical consistency across CPU/GPU backends, and tests checkpoint saving and loading.

3.1 Synthetic Benchmark Functions

The candidate optimizer is compared against reference baselines (such as AdamW [5] in FP32 Master and Native BF16 precision) across five standard test functions [16]:

B1. Ill-Conditioned Quadratic Bowl ($\kappa \geq 1000$):

$$f(w_0, w_1) = \frac{1}{2}\left(w_0^2 + 1000 w_1^2\right), \quad w^{(0)} = [10.0, 1.0]^\top. \tag{2}$$

Evaluation Focus: Tests whether the optimizer dampens oscillations along the steep axis w_1 while maintaining steady progress along the flat axis w_0.

B2. Non-Convex Rosenbrock Curved Valley [16, 17]:

$$f(w_0, w_1) = (1 - w_0)^2 + 100(w_1 - w_0^2)^2, \quad w^{(0)} = [-1.5, 2.0]^\top. \tag{3}$$

Evaluation Focus: Tests trajectory tracking along a curved valley where gradient direction changes continuously.

B3. Stochastic Gradient Noise Resilience: Evaluates the quadratic bowl under additive Gaussian gradient noise $\tilde{g}_t = \nabla f(w_t) + \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 1.0$. **Evaluation Focus:** Measures steady-state error near the optimum under noisy gradients.

B4. Multimodal Rastrigin Local-Minima Escape [18]:

$$f(w_0, w_1) = 20 + \sum_{i=0}^{1} \left(w_i^2 - 10\cos(2\pi w_i) \right), \quad w^{(0)} = [2.5, 2.5]^\top. \tag{4}$$

Evaluation Focus: Evaluates whether momentum helps the iterate move past shallow local basins toward $(0, 0)$.

B5. High-Dimensional Saddle Point Escape ($N \geq 100$):

$$f(w) = \frac{1}{2} \sum_{i=1}^{N} \lambda_i w_i^2 + \frac{1}{4} \sum_{i=1}^{N} w_i^4, \quad \lambda_i \in [-1.0, 1000.0]. \tag{5}$$

Evaluation Focus: Evaluates escape speed from saddle points with negative curvature eigenvalues.

3.2 Hyperparameter Sensitivity & Balance Calibration

To verify that an optimizer does not rely on fragile tuning, Stage II requires three hyperparameter checks:

P1. One-at-a-Time Sweep Protocol: Hold standard defaults fixed and vary each hyperparameter across at least 5 values spanning an order of magnitude.

P2. Analytical Balance Point: When a coefficient balances two competing behaviors (such as spike damping vs. sign-reversal response), derive the balance point analytically when possible and validate parameter bounds in code.

P3. Parameter Stability Range: Around the chosen default, convergence step count on the $\kappa = 1000$ benchmark should degrade by no more than 15% under a $\pm 30\%$ perturbation of auxiliary hyperparameters.

3.3 Feature Combinations, Single-Pass Fusion & Cross-Backend Parity

When an optimizer library offers multiple configuration modes and hardware backends, Stage II checks three consistency requirements:

A1. Feature Combination Stability: Across all supported combinations of buffer modes, step-clipping modes [4], and masking options [10], updates must remain finite and convergent.

A2. Single-Pass Kernel Fusion Parity (When Applicable): If an implementation fuses global norm or mask reductions into a single memory pass [19, 20] using exponential moving average (EMA) statistics from step $t - 1$, the convergence step count on $\kappa = 1000$ must not degrade by more than $+1.0\%$ relative to exact two-pass instantaneous reduction.

A3. Cross-Backend Numerical Parity:

- *Parallel Reduction Parity:* Multithreaded reductions must use deterministic pairwise or tree summation so results match single-threaded scalar execution (`atol` $= 0.0$).
- *PRNG & Rounding Parity:* Vectorized SIMD PRNG and stochastic rounding implementations must match the scalar reference bit-for-bit (`atol` $= 0.0$).
- *Multi-Step Trajectory Parity ($N = 131{,}072$, 5 steps):* Across supported precisions (FP32, BF16, FP16), vectorized CPU and GPU compute backends must match the scalar reference within atol $\leq 10^{-5}$ (FP32) and atol $\leq 2 \times 10^{-2}$ (BF16/FP16).
- *Frontend Binding Parity:* NumPy and PyTorch wrapper interfaces must produce identical parameter updates given identical inputs.

3.4 State Cleanup & Checkpoint Serialization Audit

To prevent resume bugs or temporary buffer leaks in saved checkpoints, the optimizer must pass the **Checkpoint Roundtrip Test** (Listing 2) across `float32` and `bfloat16` models: (1) no temporary staging tensors remain in `state_dict()`, (2) temporary per-step accumulators are zeroed out on export while persistent state tensors keep their expected dtypes, and (3) training resumed from a saved checkpoint matches an uninterrupted run bit-for-bit (`atol` $= 0.0$).

Listing 2: Standard LSO-Eval Stage II: Checkpoint Roundtrip & State Cleanup Test (v1.1.0)

```python
import copy, io, torch, torch.nn as nn

def verify_checkpoint_integrity(optimizer_cls):
    torch.manual_seed(42)
    forbidden_keys = (
        "p_contig", "exp_avg_contig", "exp_avg_sq_contig",
        "mp_contig", "_plan", "_contig_step"
    )
    for dtype in (torch.float32, torch.bfloat16):
        for buf in (1, 2):
            for mw in (False, "semi", "full"):
                model_cont = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 2)).to(dtype=dtype)
                model_res = copy.deepcopy(model_cont)
                opt_cont = optimizer_cls(model_cont.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                opt_res = optimizer_cls(model_res.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                batches = [(torch.randn(4, 10, dtype=dtype), torch.tensor([0, 1, 0, 1])) for _ in range(6)]

                for i in range(5):
                    x, y = batches[i]
                    for m, opt in ((model_cont, opt_cont), (model_res, opt_res)):
                        loss = nn.CrossEntropyLoss()(m(x).float(), y)
                        opt.zero_grad(); loss.backward(); opt.step()

                buffer = io.BytesIO()
                torch.save(opt_res.state_dict(), buffer); buffer.seek(0)
                opt_loaded = optimizer_cls(model_res.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                state_dict = torch.load(buffer, weights_only=False)
                opt_loaded.load_state_dict(state_dict)

                for p_state in state_dict["state"].values():
                    for k in forbidden_keys:
                        assert k not in p_state, f"Memory leak: '{k}' in state_dict!"
                    if "scratch" in p_state:
```

```
                        sc = p_state["scratch"]
                        assert float(sc[0]) == 0.0 and float(sc[1]) == 0.0
                        assert float(sc[2]) == 0.0 and float(sc[5]) == 0.0

                for p in model_res.parameters():
                    st = opt_loaded.state[p]
                    if "scratch" in st: assert st["scratch"].dtype == torch.float32
                    if "master_param" in st: assert st["master_param"].dtype ==
    torch.float32

                x_last, y_last = batches[5]
                for m, opt in ((model_cont, opt_cont), (model_res, opt_loaded)):
                    loss = nn.CrossEntropyLoss()(m(x_last).float(), y_last)
                    opt.zero_grad(); loss.backward(); opt.step()

                for p_c, p_r in zip(model_cont.parameters(), model_res.parameters()):
                    torch.testing.assert_close(p_c, p_r, rtol=0.0, atol=0.0)
    return True
```

Reference Case Example: LuminaV (v1.5.3) Trajectory, Equilibrium & Backend Parity in Stage II

Proposition 1 (Symmetric Spike–Reversal Balance for λ)**.** *Let $w_1 = \beta_1^2$ and $w_2 = 1 - \beta_1^2$ with $\beta_1 \in (\frac{\sqrt{3}}{2}, 1)$. In TD-CLM [2], the velocity ceiling under a large gradient spike ($|g_t| \to \infty$) is $u_{spike}(\lambda) = \frac{w_2}{w_2+\lambda}$, and the velocity ceiling under an exact gradient sign reversal ($g_t = -m_{t-1}$) is $u_{rev}(\lambda) = \frac{w_1-w_2}{1+2\lambda}$. The coefficient λ^* that equalizes spike damping and reversal damping ($u_{spike}(\lambda^*) = u_{rev}(\lambda^*)$) is:*

$$\lambda^* = \frac{2w_2^2}{w_1 - 3w_2} = \frac{2(1-\beta_1^2)^2}{4\beta_1^2 - 3}, \quad \forall \beta_1 \in \left(\frac{\sqrt{3}}{2}, 1\right). \tag{6}$$

Proof. Setting $\frac{w_2}{w_2+\lambda} = \frac{w_1-w_2}{1+2\lambda}$ gives $w_2(1+2\lambda) = (w_1 - w_2)(w_2 + \lambda)$. Expanding and collecting terms in λ yields $w_2(1 - w_1 + w_2) = (w_1 - 3w_2)\lambda$. Substituting $1 - w_1 = w_2$ gives $2w_2^2 = (w_1 - 3w_2)\lambda$, which solves to $\lambda^* = \frac{2(1-\beta_1^2)^2}{4\beta_1^2-3}$ for $4\beta_1^2 - 3 > 0 \iff \beta_1 > \frac{\sqrt{3}}{2} \approx 0.866$. For $\beta_1 = 0.9$, $\lambda^* \approx 0.3008 \approx 0.30$. □

LuminaV Architecture-Specific Parity Details (`luminav-core`, `luminav-simd`, `luminav-vulkan`):

- **1-Pass 6-Slot Scratchpad EMA:** In `fused_single_pass=True`, each parameter maintains a 6-element `float32` scratchpad $\mathbf{s} = [\Sigma m_{\text{mask}}, \Sigma u_{\text{eff}}^2, \Sigma \theta_0^2, \gamma_{\text{cautious}}, S_{\text{bound}}, 0.0] \in \mathbb{R}^6$. The kernel reads s_3, s_4 from step $t-1$ while accumulating s_0, s_1, s_2 in a single memory pass, updating s_3, s_4 post-step via EMA ($\alpha_{\text{ema}} = 0.95$ for Buffer 1; 0.80 for Buffer 2). On $\kappa = 1000$, Buffer 1 1-Pass Fused EMA converged in **346 steps** vs. 381 steps for 2-Pass Instantaneous ($-$**9.19**%), and Buffer 2 converged in **367 steps** vs. 370 steps ($-$**0.81**%).
- **Synthetic Surfaces (`B1`–`B5`):**
  - *B1 Quadratic ($\kappa = 1000$, $\eta = 0.02$, 1,000 steps):* Unregularized B-CLM ($\lambda = 0.0$) required 950 steps, whereas **LuminaV Buffer 1 (TD-CLM)** converged in **575 steps** (loss gap 2.67×10^{-6}) and **LuminaV Buffer 2 (Dual)** in **571 steps** (9.31×10^{-6}). Both `AdamW_FP32_Master` (gap 3.66×10^{-2}) and

`AdamW_Native_BF16` (gap 50.00) did not reach 10^{-5} within 1,000 steps.

- *B2 Rosenbrock, B3 Noise ($\sigma = 1.0$), B4 Rastrigin & B5 Saddle ($N = 100$):* On Rosenbrock, Buffer 1 reached 7.46×10^{-4} and Buffer 2 reached 1.76 vs. 3.90 for `AdamW_FP32_Master`. On $N = 100$ Saddle Escape, **LuminaV Buffer 2** converged in **149 steps** vs. 193 steps for `AdamW_FP32_Master`. Under Gaussian noise ($\sigma = 1.0$), Buffer 2 (8.11×10^{-2}) and Buffer 1 (2.21×10^{-1}) achieved lower steady-state loss than `AdamW_FP32_Master` (3.36).

- **Cross-Backend & Checkpoint Parity:** Multithreaded Rayon chunking (`ELEM_CHUNK_SIZE` = 2,097,152 with 16-lane pairwise tree reduction), AVX-512/AVX2 SIMD, Vulkan 1.3/1.4 Wave32 UMA shaders, all 12 feature ablation combinations, and all 24 checkpoint roundtrip configurations passed within specification bounds.

Listing 3: LuminaV Reference Stage II: 2D Trajectory Simulation Script (eval/modules/benchmarks_2d.py)

```python
import numpy as np

def run_2d_benchmark(loss_fn, grad_fn, step_fn, w_init, f_star=0.0,
                     lr=0.01, max_steps=2000, tol=1e-5, noise_std=0.0, seed=1337):
    rng = np.random.default_rng(seed)
    w = np.array(w_init, dtype=np.float64)
    state = {}
    current_loss = loss_fn(w)

    for t in range(1, max_steps + 1):
        g = grad_fn(w)
        if noise_std > 0.0:
            g = g + rng.normal(0.0, noise_std, size=g.shape)
        w, state = step_fn(w, g, state, lr, t)
        current_loss = loss_fn(w)
        curr_gap = abs(current_loss - f_star)
        if np.isnan(w).any() or np.isinf(w).any() or curr_gap > 1e8:
            return t, curr_gap, "DIVERGED"
        if curr_gap < tol:
            return t, curr_gap, "CONVERGED"
    return max_steps, abs(current_loss - f_star), "MAX_STEPS"

def luminav_tdclm_step_fn(w, g, state, lr, t,
                          beta1=0.9, lam=0.3, eps=1e-8, bound_ratio=0.05):
    if "m" not in state:
        state["m"] = np.zeros_like(w)
    m = state["m"]
    w1 = beta1 * beta1
    w2 = 1.0 - w1
    m_new = beta1 * m + (1.0 - beta1) * g
    clm_m = w1 * m + w2 * g
    diff = g - m
    clm_env = w1 * np.abs(m) + w2 * np.abs(g) + lam * np.abs(diff) + eps
    u = clm_m / clm_env
    mask = (u * g > 0.0).astype(np.float64)
    cautious_scale = 1.0 / np.clip(np.mean(mask), 0.5, 1.0)
    step_val = lr * (u * mask) * cautious_scale
    b_i = bound_ratio * np.sqrt(w * w + 1e-4)
    step_val = b_i * np.tanh(step_val / b_i)
    state["m"] = m_new
    return w - step_val, state
```

4 Stage III: Short-Run Proxy Model Screening

Stage III evaluates candidate configurations on a small proxy neural network before committing to large-scale training runs. Short proxy runs help filter out unstable learning rates and detect low-precision rounding issues early.

4.1 Proxy Model Selection & Step Budget

The evaluation uses a small proxy model representative of the target architecture (for example, a 4-layer causal Transformer with RoPE, SwiGLU MLP, and RMSNorm):

M1. Step Limit: The screening run is capped at 5% to 15% of the full training budget (typically 500 to 2,000 optimization steps).

M2. Batch Size Diversity: Evaluate at least two effective batch sizes (1× micro-batch and 4× gradient-accumulated batch) to check stability under different gradient noise levels.

M3. Controlled Schedule: Use an identical linear warmup (5% of steps) followed by cosine decay to 5% of peak learning rate across all candidates.

4.2 Early Stopping & Rejection Criteria

A candidate configuration is rejected if any of the following three failure modes occur:

F1. Update-to-Weight Ratio Breach: Compute the layer-wise relative update norm across weight matrices (ndim ≥ 2 and `param.squeeze().ndim` ≥ 2, excluding 1D biases, normalization scales, and singleton-broadcasted vectors):

$$R_{t,l} = \frac{\|\delta_{t,l}\|_2}{\|\theta_{t,l}\|_2}. \tag{7}$$

Rejection Rule: If $R_{t,l} > 0.10$ for any matrix layer l after warmup (excessive step size) or $R_{t,l} < 10^{-7}$ during active training ($t > t_{\text{warmup}}$ and $\eta_t \geq 0.1\eta_{\max}$, stalled updates), the configuration is rejected.

F2. Unrecovered Loss Spikes: If the exponential moving average (EMA, $\alpha = 0.99$) training loss jumps by more than 0.5 nats above its pre-spike minimum and does not recover within 50 consecutive steps, the run is rejected.

F3. BF16/FP16 Rounding Underflow: Compare the final EMA training loss of the native low-precision (`bfloat16` or `float16`) proxy trial against an identical reference trial executed in `float32` precision [14, 15]:

$$\Delta\mathcal{L}_{\text{stall}} = \mathcal{L}_{\text{EMA}}^{(\text{BF16/FP16})}(T) - \mathcal{L}_{\text{EMA}}^{(\text{FP32-Ref})}(T). \tag{8}$$

Rejection Rule: If $\Delta\mathcal{L}_{\text{stall}} \geq 0.25$ nats, the low-precision update suffers from severe mantissa truncation and is rejected.

4.3 Learning Rate & Clipping Grid

Engineers evaluate a learning rate grid ($\eta \in [3 \times 10^{-4}, 10^{-2}]$) to identify:

- **Long-Run Setting ($>$ 20k steps):** Conservative learning rates ($\eta \in [3 \times 10^{-4}, 8 \times 10^{-4}]$, $\lambda_{\text{wd}} = 8 \times 10^{-2}$) suitable for long-horizon pre-training.
- **Short-Run Setting ($<$ 10k steps):** Higher learning rates ($\eta = 10^{-2}$) tested with coordinate or radial step clipping [4].

Reference Case Example: LuminaV (v1.5.3) Proxy Model Screening in Stage III

Stage III screening was run on `LuminaV v1.5.3` using `MProxyLLM` (4-layer causal Transformer with RoPE, SwiGLU, and RMSNorm; $V = 2{,}048, d_{\text{model}} = 256, n_{\text{head}} = 4, d_{\text{ffn}} = 682$; 500 steps, 25-step warmup, batch size 8, sequence length 128) across 1× and 4× gradient accumulation:

Configuration	BF16 EMA	FP32 Ref	Max $R_{t,l}$	Min $R_{t,l}$	F_1	F_2	F_3 (Verdict)
LuminaV Buf 1 (1×, **LR** 8×10^{-4})	0.9018	0.9021	0.01574	1.24×10^{-3}	PASS	PASS	PASS (**PASSED**)
LuminaV Buf 1 (4×, **LR** 8×10^{-4})	**0.8462**	**0.8462**	0.01602	1.24×10^{-3}	PASS	PASS	PASS (**PASSED**)
LuminaV Buf 2 (1×, **LR** 8×10^{-4})	0.9146	0.9141	0.01604	1.03×10^{-3}	PASS	PASS	PASS (**PASSED**)
LuminaV Buf 2 (4×, **LR** 8×10^{-4})	0.8518	0.8515	0.01684	9.81×10^{-4}	PASS	PASS	PASS (**PASSED**)
LuminaV Buf 1 (1×, **LR** 1×10^{-2})	0.9326	0.9317	0.04041	3.03×10^{-3}	PASS	PASS	PASS (**PASSED**)
AdamW Native BF16 (1×, LR 8×10^{-4})	0.9763	0.9552	0.01423	7.76×10^{-5}	PASS	PASS	PASS (PASSED)
AdamW Native BF16 (4×, LR 8×10^{-4})	0.8767	0.8678	0.01600	7.50×10^{-5}	PASS	PASS	PASS (PASSED)
AdamW FP32 Master (1×, LR 3×10^{-4})	1.0989	1.0992	0.00611	1.00×10^{-4}	PASS	PASS	PASS (PASSED)
AdamW FP32 Master (4×, LR 3×10^{-4})	0.9693	0.9692	0.00708	1.00×10^{-4}	PASS	PASS	PASS (PASSED)

Summary: All five LuminaV configurations kept $|\mathcal{L}_{\text{BF16}} - \mathcal{L}_{\text{FP32}}| \leq 0.0009$ nats without FP32 master weights, compared to a +0.0211 nat BF16 gap on Native BF16 AdamW. At $\eta = 10^{-2}$, coordinate bounding (`bound_ratio=0.05`) kept $\max R_{t,l} = 0.04041 \leq 0.10$.

5 Stage IV: Large-Scale Pre-Training Evaluation

Stage IV evaluates candidate optimizers on large-scale language model pre-training across two schedules:

1. **Stage IV-A (Medium-Length Hardware & Burn-in Run):** Runs of 2,500 to 5,000 steps at context length 512 with gradient accumulation (8×, 4,096 tokens/step), measuring isolated optimizer VRAM, step latency, and held-out validation perplexity.
2. **Stage IV-B (Full-Schedule & Long-Context Run):** Longer training runs ($\geq$ 20,000 steps, context length $\geq$ 2,048) evaluating long-horizon learning rate decay and validation perplexity.

5.1 Streaming Data Hygiene, Domain Interleaving & Sequence Packing

To prevent overlap between streaming training data and the held-out evaluation set, avoid sequential domain-block bias, and eliminate GPU padding waste, Stage IV applies six data pipeline rules prior to and during training:

D1. Document Hash Logging (XXH3-64 WAL): All documents used to build the held-out validation set ($N_{\text{eval}} \geq 450{,}000$ tokens) are hashed with 64-bit XXH3 and recorded in a Write-Ahead Log (`.dedup` WAL).

D2. Cross-Context Token Multiset Check: When evaluating across multiple sequence lengths (such as $880 \times 512 = 450{,}560$ tokens in Stage IV-A and $220 \times 2{,}048 = 450{,}560$

tokens in Stage IV-B), the sorted token multisets are checked to ensure they contain the exact same tokens.

D3. Stream Offset & Deduplicator Preloading: Training streams skip the raw records used for validation (`skip_raw_items`) and preload the validation WAL hashes so validation documents are excluded during training.

D4. Weighted Multi-Domain Interleaving (No Block Concatenation): When training on K heterogeneous sources or domains $D_1, \ldots, D_K$ with target mixture weights $\mathbf{p} = (p_1, \ldots, p_K)$ ($\sum_{k=1}^{K} p_k = 1$), streams must be merged via seeded weighted interleaving (e.g., deterministic multinomial stream selection or per-window domain quotas) rather than sequential dataset concatenation. This keeps the domain mixture stationary across warmup, peak learning rate, and cosine decay.

D5. Two-Level Streaming Shuffle: Before tokenization and packing, streaming loaders apply *Level-1 macro-shuffling* (permuting shard/file order via `seed + epoch`) followed by *Level-2 micro-shuffling* through an in-memory reservoir shuffle buffer ($B_{\text{shuf}} \geq 10{,}000$ documents, pre-filled before yielding step 1) so adjacent documents inside a packed sequence originate from decorrelated shards and domains.

D6. EOD-Delimited Sequence Packing & Asynchronous GPU Transfer: Rather than padding short documents with `<PAD>` tokens (which wastes quadratic attention FLOPs and skews effective batch sizes), tokenized documents popped from the shuffle reservoir are terminated with an explicit End-of-Document token (`EOD`/`EOS`, e.g., `<|endoftext|>`) and concatenated into a continuous token buffer sliced into dense (B, L) blocks with zero padding (100% token utilization; optionally passing cumulative document offsets `cu_seqlens` and reset `position_ids` when using document-masked FlashAttention `varlen`). Packed batches are prefetched in page-locked host memory (`pin_memory=True`, `persistent_workers=True`) and transferred via `.to(device, non_blocking=True)` so host-to-device DMA overlaps with GPU compute.

5.2 Hardware Profiling & Isolated Optimizer Memory Footprint

All candidate and baseline optimizers run on the same hardware configuration with matched random seeds, data order, and sequence lengths. Following prior work on memory-efficient optimizers (8-bit Optimizers [21] and GaLore [9]), Stage IV separates peak active PyTorch tensor memory (`torch.cuda.max_memory_allocated()`) into the fixed non-optimizer baseline $\text{VRAM}_{\text{base}}$ and the **Isolated Optimizer Footprint** VRAM_{opt} (Table 2):

$$\text{VRAM}_{\text{peak}} = \underbrace{W_{\text{model}} + G_{\text{grad}} + A_{\text{ctx}}}_{\text{Fixed Model \& Activation Baseline } (\text{VRAM}_{\text{base}})} + \underbrace{\text{VRAM}_{\text{opt}}}_{\text{Isolated Optimizer State \& Master Copy}} . \quad (9)$$

Here `max_memory_allocated()` measures active tensor bytes allocated by PyTorch (excluding cached allocator pools and CUDA/Triton runtime context). Because $\text{VRAM}_{\text{base}}$ is fixed by the model and context length, $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ reflects the memory used by the optimizer states and any master weight copies [21, 9].

Table 2: Stage IV: Standard Evaluation Metrics

Dimension	Metric	Measurement Method & Tooling
1. Convergence	Raw Loss & EMA Loss	Recorded at every step; smoothed via EMA ($\alpha \in \{0.99, 0.995\}$).
2. Generalization	Held-Out Validation PPL	Evaluated on $\geq$ 450k deduplicated validation tokens ($\exp(\mathcal{L}_{\text{val}})$).
3. Memory	$\text{VRAM}_{\text{peak}}$ & VRAM_{opt} (GB)	Active tensor peak via `max_memory_allocated()` [22] and isolated $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ [21, 9].
4. Latency	Step & Opt Kernel Time (ms)	End-to-end step time and isolated CUDA Event timer around `opt.step()`.

5.3 Two-Class Acceptance Criteria

Stage IV evaluates checkpoints on the held-out validation set via perplexity:

$$\text{PPL}_{\text{held-out}} = \exp\left(\frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_{\text{CE}}(x_i)\right). \tag{10}$$

Because single-buffer ($\approx$ 2 B/param) and dual-buffer (4–12 B/param) optimizers are designed for different memory budgets [21, 9], LSO-Eval v1.1.0 defines **Two-Class Acceptance Criteria** relative to the Native BF16 AdamW baseline [5] and FP32 Master AdamW reference:

Class I. Dual-Buffer / Full-State Optimizers ($\geq$ 4 bytes/param): The candidate maintains both first- and second-moment states (m_t, v_t). It passes Class I if it achieves **equal or lower held-out validation perplexity** than Native BF16 AdamW at matched token count ($\Delta\text{PPL}_{\text{held-out}} = \text{PPL}_{\text{cand}} - \text{PPL}_{\text{AdamW-BF16}} \leq 0.0$) while passing $\text{F}_1 \wedge \text{F}_2$ stability checks without using additional optimizer memory ($\Delta\text{VRAM}_{\text{opt}} \leq 0.0$ GB).

Class II. Single-Buffer / Compressed-State Optimizers ($\leq$ 2 bytes/param): The candidate omits the second-moment buffer v_t (or compresses state tensors) to reduce memory usage on constrained GPUs. It passes Class II if it satisfies four practical conditions:

(a) *Dynamic Stability ($F_1 \wedge F_2$):* Layer-wise update ratios remain within $R_{t,l} \in [10^{-7}, 0.10]$ without unrecovered loss spikes.

(b) *Validation Loss Stability:* Held-out validation perplexity improves across evaluation checkpoints without late-stage divergence.

(c) *Memory & Step Efficiency:* Reduces isolated optimizer memory by at least **35.0**% ($\Delta\text{VRAM}_{\text{opt}} \leq -35.0\%$) relative to Native BF16 AdamW and at least **70.0**% ($\Delta\text{VRAM}_{\text{opt}} \leq -70.0\%$) relative to FP32 Master AdamW[1], while keeping per-step optimizer latency within 1.15$\times$ of Native BF16 AdamW and faster than FP32 Master

[1] In pure 1-buffer `bfloat16` mode (2 B/param) on multi-billion-parameter models where fixed allocator alignment is negligible, the theoretical reduction reaches -50.0% vs. Native BF16 AdamW (4 B/param) and -83.3% vs. FP32 Master AdamW (12 B/param). Setting the acceptance thresholds at -35.0% and -70.0% accommodates allocator block rounding on smaller models and hybrid/subspace compression schemes.

AdamW.

(d) *Bounded Compression Loss Gap:* The validation log-perplexity difference relative to two-buffer Native BF16 AdamW remains within the Stage III F_3 tolerance:

$$\Delta\mathcal{L}_{\text{val}} = \ln\left(\frac{\text{PPL}_{\text{cand}}}{\text{PPL}_{\text{AdamW-BF16}}}\right) \leq 0.25 \text{ nats.} \tag{11}$$

Reference Case Example: LuminaV (v1.5.3) 4B-Parameter Hybrid LLM Evaluation in Stage IV-A & IV-B

1. Model Architecture & Deduplicated Data Pipeline: The Stage IV-A and IV-B runs below evaluated **LuminaV** `v1.5.3` (`centered_defect=False`, native `torch.bfloat16`) from random initialization on the hybrid **Qwen/Qwen3.5-4B-Base** architecture (32 layers: 24 GatedDeltaNet linear-attention layers + 8 full SDPA layers; $\approx 4.26 \times 10^9$ parameters, fixed model weight + gradient footprint $W_{\text{BF16}} + G_{\text{BF16}} = 15.88$ GB). Under FSDP (`fsdp_sync=True`), LuminaV packs local reduction scalars $[\Sigma m_{\text{mask}}, \Sigma u_{\text{eff}}^2, \Sigma\theta_0^2, N_{\text{local}}]$ into a single asynchronous `all_reduce` call per parameter group. The data loader streamed from a 6-way interleaved English–Indonesian corpus with XXH3-64 deduplication:

- *Validation Tensor Consistency:* Stage IV-A ($880 \times 512 = 450{,}560$ tokens, SHA-256 `4d278cb87e4abbe1`) and Stage IV-B ($220 \times 2{,}048 = 450{,}560$ tokens, SHA-256 `108e517e1c181cb6`) consumed 622 raw stream records (`raw_offset`) with identical token multisets.
- *Deduplication WAL Continuity:* All 143,475 XXH3-64 validation hashes from Stage IV-A were preloaded into the 295,605 committed hashes of the Stage IV-B WAL so validation records were skipped during training.
- *Sampling Schedule Note (`v1.5.3`):* In the `v1.5.3` loader script, parallel records were filtered by joint `(source, domain)` pairs rather than per-domain quotas; because all candidate and baseline optimizers used the exact same deterministic stream order, relative cross-optimizer comparisons remain controlled.

2. Stage IV-A Results (Context 512, Grad Accum 8×, 4,800 Steps / 19.66M Tokens): At sequence length 512, the fixed non-optimizer baseline (weights + gradients + activations) is $\text{VRAM}_{\text{base}}^{(512)} \approx 20.87$ GB. Across the 4,800-step run evaluated on 450,560 held-out tokens:

Optimizer Configuration	Final EMA	Val PPL	ΔPPL	Peak VRAM	Opt VRAM (VRAM_{opt})	Max $R_{t,l}$	Step Time	Opt Time	LSO-Eval Status
LuminaV Buffer 2 (Dual, 8×10^{-4})	**4.7621**	**141.96**	**−39.21**	36.75 GB	15.88 GB (0.0%)	0.0047	841.6 ms	48.37 ms	**PASSED (Class I)**
LuminaV Buffer 1 (TD-CLM, 3×10^{-4})	5.2569	195.58	+14.41	**28.81** GB	**7.94** GB (**−50.0**%)	0.0120	**829.4** ms	36.77 ms	**PASSED (Class II)**
LuminaV Buffer 1 (TD-CLM, 8×10^{-4})	5.2953	221.07	+39.90	**28.81** GB	**7.94** GB (**−50.0**%)	0.0191	830.7 ms	**36.46** ms	**PASSED (Class II)**
AdamW (Native BF16, 3×10^{-4}) [5]	5.4497	181.17	+0.00	36.75 GB	15.88 GB (Ref)	0.0034	835.4 ms	41.58 ms	Baseline (BF16)
AdamW (FP32 Master, 3×10^{-4}) [5]	5.0234	153.78	−27.39	75.36 GB	54.49 GB (+243%)	0.0040	919.5 ms	123.35 ms	Baseline (Master)

3. Stage IV-B Results (Context 2,048, Grad Accum 1×, 20,000 Steps / 40.96M Tokens): At sequence length 2,048, autograd activations increase the fixed baseline to $\text{VRAM}_{\text{base}}^{(2048)} \approx 28.03$ GB, while isolated optimizer state memory remains **7.94** GB (Buffer 1) and **15.86** GB (Buffer 2):

Optimizer Configuration	Final EMA	Val PPL	ΔPPL	Peak VRAM	Opt VRAM (VRAM_{opt})	Max $R_{t,l}$	Step Time	Opt Time	LSO-Eval Status
LuminaV Buffer 2 (Dual, 8×10^{-4})	**4.5549**	**89.14**	**−43.43**	43.89 GB	15.86 GB (0.0%)	0.0189	297.1 ms	48.31 ms	**PASSED (Class I)**
LuminaV Buffer 1 (TD-CLM, 3×10^{-4})	4.9173	165.13	+32.56	**35.97** GB	**7.94** GB (**−50.0**%)	0.0087	**283.5** ms	**36.99** ms	**PASSED (Class II)**
AdamW (Native BF16, 3×10^{-4}) [5]	4.7621	132.57	+0.00	43.89 GB	15.86 GB (Ref)	0.0037	288.0 ms	41.48 ms	Baseline (BF16)
AdamW (FP32 Master, 3×10^{-4}) [5]	4.8116	124.43	−8.14	80.11 GB	52.08 GB (+228%)	0.0045	370.1 ms	123.33 ms	Baseline (Master)

Summary of Findings:

1. **Class I (LuminaV Buffer 2):** Without FP32 master weights, `LuminaV Buffer 2 (Dual)` reached the lowest validation perplexity in both Stage IV-A (**141.96** vs. 181.17 on Native BF16 AdamW and 153.78 on FP32 Master AdamW) and Stage IV-B (**89.14** vs. 132.57 on Native BF16 AdamW and 124.43 on FP32 Master AdamW, while using 36.22 GB less VRAM than FP32 Master AdamW).
2. **Class II (LuminaV Buffer 1):** `LuminaV Buffer 1 (TD-CLM, LR` 3×10^{-4}`)` reduced isolated optimizer state memory by **50.0**% (7.94 GB vs. 15.88 GB on BF16 AdamW, exceeding the −35.0% Class II requirement) and by **84.8**%–**85.4**% compared to FP32 Master AdamW (52.08–54.49 GB, exceeding the −70.0% requirement and allowing 4B pre-training to fit on a single 40 GB GPU). Its log-perplexity gap remained within the Class II bound ($\Delta\mathcal{L}_{\text{val}} = 0.0765 \leq 0.25$ nats in IV-A and $0.2196 \leq 0.25$ nats in IV-B).

6 V-Eval: Vision Modality Evaluation Protocol

Stages I and II test mathematical and numerical properties, while Stages III and IV evaluate language models. Computer vision models introduce different optimization dynamics: convolutional stems, batch and response normalization, normalization-free networks, vision transformers under strong image augmentation, and flow-matching generative models each respond differently to momentum and step clipping. **V-Eval** is a standalone vision evaluation protocol designed to test these architectures across multiple image resolutions without requiring costly full ImageNet-1K (1.28M-image) training runs.

6.1 Model Sources and Two-Track Structure

To keep comparisons reproducible, V-Eval loads standard models from established public libraries—`timm` (PyTorch Image Models), `torchvision.models`, Hugging Face `transformers`, and the official `facebookresearch/SiT` repository.

V-Eval is divided into two tracks:

1. **Track V-A (Core — Vision From-Scratch Training):** Trains five standard vision architectures (`V-A1` through `V-A5`) from random initialization (`pretrained=False`, `seed=42`) in `torch.bfloat16` across three spatial resolution regimes (32×32, 128×128, and 224×224).
2. **Track V-B (Optional — Vision Downstream Fine-Tuning):** Evaluates fine-tuning of pretrained backbones on object detection (`V-B1`) and fine-grained classification (`V-B2`) when additional compute budget is available. This track tests whether an optimizer preserves pretrained features without rounding underflow at small learning rates ($\eta \leq 10^{-4}$) in `bfloat16`.

6.2 Six-Optimizer Comparison Matrix and Parameter Grouping

To compare native `bfloat16` training (without master weights) against `float32` master-weight training across both two-buffer and single-buffer optimizers, every model in Track V-A and Track V-B is evaluated across six configurations:

1. `LuminaV_Buffer2_Dual` **(Class I Candidate — 2-Buffer Native BF16):** `buffer=2`, `tau=0.8`, `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08`, `bound=True`, `bound_type="coordinate"`, `bound_ratio=0.05`, `master_weights="none"`, `stochastic_rounding=True`, `fused_single_pass=True`.
2. `LuminaV_Buffer1_TDCLM` **(Class II Candidate — 1-Buffer Native BF16):** `buffer=1`, `lam=0.3`, `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08`, `bound=True`, `bound_type="coordinate"`, `bound_ratio=0.05`, `master_weights="none"`, `stochastic_rounding=True`, `fused_single_pass=True`.
3. `AdamW_Native_BF16` **(Baseline 2-Buffer — Native BF16):** `torch.optim.AdamW` [5], `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08` (From-Scratch) / `0.05` (Fine-Tuning), `fused=True`, `use_fp32_master=False`.
4. `AdamW_FP32_Master` **(Reference 2-Buffer — BF16 + FP32 Master Copy):** Same hyperparameters as `AdamW_Native_BF16`, with an explicit `float32` master weight copy (`use_fp32_master=True`).
5. `Lion_Native_BF16` **(Baseline 1-Buffer — Native BF16):** `SimpleLion` [3] (Listing 4), `betas=(0.9, 0.99)`, `weight_decay=0.8` (matching effective weight decay $\eta\lambda = 8 \times 10^{-5}$ in Track V-A), `use_fp32_master=False`.
6. `Lion_FP32_Master` **(Reference 1-Buffer — BF16 + FP32 Master Copy):** Same hyperparameters as `Lion_Native_BF16`, with an explicit `float32` master weight copy (`use_fp32_master=True`).

Weight Decay Parameter Grouping (`split_vision_parameter_groups`): For all six optimizers (as well as LLM/hybrid runs in Stages III and IV), model parameters are split into two groups before training (Listing 4):

- **Decayed Group ($\lambda_{\text{wd}} > 0$):** All weight matrices and convolutional kernels with ndim ≥ 2 and `param.squeeze().ndim` ≥ 2, including standard and depthwise convolutions (`nn.Conv2d`), linear projections (`nn.Linear`), QKV/MLP matrices, and vocabulary token embeddings (unless excluded via `model.no_weight_decay()`).
- **No-Decay Group ($\lambda_{\text{wd}} = 0.0$):**
  (a) *$0D$/$1D$ and Singleton-Broadcasted $1D$ Tensors (*ndim ≤ 1 *or* `param.squeeze().ndim` ≤ 1*):* All biases, 1D affine scales/shifts, and channel- or token-wise vectors stored with singleton broadcast dimensions (such as $(1, 1, 1, C)$ `GRN` scales in ConvNeXt-V2, $(1, C, 1, 1)$ channel scales, and $(1, 1, D)$ single-token embeddings).
  (b) *Normalization & Normalization-Free Scaling Layers:* Parameters of `BatchNorm`, `SyncBatchNorm`, `LayerNorm`, `RMSNorm` (including custom LLM/Vision `RMSNorm` and attention `QK-Norm` `q_norm`/`k_norm`), `GroupNorm`, `InstanceNorm`, `LocalResponseNorm`, `GlobalResponseNorm` (`GRN`), `LayerScale` (γ, `ls1`, `ls2`), and Dynamic Tanh (`DyT` α) or NF-Net skip gains (`gain`, `skipinit`).
  (c) *Multi-Dimensional Positional, Register & Conditioning Tables (*ndim ≥ 2*):* Vision and multimodal structural tokens (`cls_token`, `reg_token`/`register_token`, `dist_token`, `mask_token`), spatial/relative positional tables (`pos_embed`, `absolute_pos_embed`, `relative_position_bias_table`, `rel_pos`, `rope`/`freqs_cis`), and detection query embeddings (`query_embed`, `query_pos`, `level_embed`).

(d) *Hybrid LLM / State-Space Model (SSM) Recurrent Parameters & Framework Hooks:* Log-decay matrices and discretization biases in Mamba, Mamba-2, and GatedDeltaNet (`A_log`, `dt_bias`, skip connection `D`), parameters marked with `_no_weight_decay = True`, and names returned by `model.no_weight_decay()`.

Baseline Parameter Alignment and Effective Weight Decay (`v1.1.1`): In LSO-Eval `v1.1.1`, the baseline `SimpleLion` optimizer for vision tasks uses $\boldsymbol{\beta} = (0.9, 0.99)$ and weight decay $\lambda = 0.8$ (Listing 4), following the vision settings in Chen et al. [3] (Section 4.1). Under decoupled weight decay ($\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\Delta_t$), the per-step shrinkage is $\eta\lambda$. Because Lion uses a smaller learning rate ($\eta_{\text{Lion}} = 10^{-4}$) than AdamW ($\eta_{\text{AdamW}} = 10^{-3}$), setting $\lambda_{\text{Lion}} = 0.8$ keeps the effective weight decay equal across optimizers:

$$\eta_{\text{Lion}} \cdot \lambda_{\text{Lion}} = 10^{-4} \times 0.8 = 8 \times 10^{-5} = 10^{-3} \times 0.08 = \eta_{\text{AdamW}} \cdot \lambda_{\text{AdamW}}. \tag{12}$$

Listing 4: Standard LSO-Eval V-Eval: Standalone SimpleLion Optimizer (Chen et al., NeurIPS 2023, Algorithm 2) & Decoupled Parameter Grouping

```python
import torch
import torch.nn as nn

class SimpleLion(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-4, betas=(0.9, 0.99), weight_decay=0.8):
        if lr < 0.0:
            raise ValueError(f"Invalid learning rate: {lr}")
        if not 0.0 <= betas[0] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 0: {betas[0]}")
        if not 0.0 <= betas[1] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 1: {betas[1]}")
        if weight_decay < 0.0:
            raise ValueError(f"Invalid weight_decay value: {weight_decay}")
        defaults = dict(lr=lr, betas=betas, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, (beta1, beta2), wd = group["lr"], group["betas"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad
                if grad.is_sparse:
                    raise RuntimeError("SimpleLion does not support sparse gradients.")
                state = self.state[p]
                if len(state) == 0:
                    state["exp_avg"] = torch.zeros_like(p,
    memory_format=torch.preserve_format)
                exp_avg = state["exp_avg"]
                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)
                update = exp_avg.lerp(grad, 1.0 - beta1).sign_()
                p.add_(update, alpha=-lr)
                exp_avg.lerp_(grad, 1.0 - beta2)
        return loss

def split_vision_parameter_groups(model: nn.Module, weight_decay: float = 0.08):
    no_decay_kw = (
```

```
        "bias", "bn", "norm", "ln", "rmsnorm", "groupnorm", "grn", "global_response_norm",
        "q_norm", "k_norm", "qk_norm", "layerscale", "ls1", "ls2", "gamma", "beta",
        "dyt", "dynamic_tanh", "alpha", "gain", "skipinit",
        "cls_token", "reg_token", "register_token", "dist_token", "mask_token",
        "pos_embed", "absolute_pos_embed", "relative_position_bias_table", "rel_pos",
        "rope", "freqs_cis", "query_embed", "query_pos", "level_embed",
        "a_log", "dt_bias", "d_skip",
    )
    explicit_no_decay = (
        set(model.no_weight_decay())
        if hasattr(model, "no_weight_decay") and callable(model.no_weight_decay)
        else set()
    )
    decay, no_decay = [], []
    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        lname = name.lower()
        if (
            param.ndim <= 1
            or param.squeeze().ndim <= 1
            or getattr(param, "_no_weight_decay", False)
            or name in explicit_no_decay
            or name == "D" or name.endswith(".D")
            or any(k in lname for k in no_decay_kw)
        ):
            no_decay.append(param)
        else:
            decay.append(param)
    return [
        {"params": decay, "weight_decay": float(weight_decay)},
        {"params": no_decay, "weight_decay": 0.0},
    ]
```

6.3 Track V-A: Vision From-Scratch Evaluation (Core)

All Track V-A models are trained from scratch (`pretrained=False`, `seed=42`) in `torch.bfloat16` on a single GPU (reference target: 1× NVIDIA A100 40 GB). Rather than restricting all tasks to 32×32 CIFAR images (which would shrink the final-stage feature maps of ImageNet-stem ConvNets to 1×1), Track V-A spans three resolution tiers using standard, compute-friendly ImageNet subsets alongside CIFAR (Table 3):

V-A1. Residual CNN (`ResNet-50` [23], CIFAR 3×3 stem, ≈ 23.7M params): Standard residual network with `BatchNorm2d` and `ReLU` trained on **CIFAR-100** (32×32, 50k train / 10k val, 100 classes) [5, 6, 11], loaded via `timm.create_model("resnet50", pretrained=False, num_classes=100)` with a CIFAR 3×3 stem.

V-A2. Normalization-Free CNN (`NF-ResNet-50` [4], ≈ 25.6M params): Replaces batch normalization with Scaled Weight Standardization (`ScaledStdConv2d`), trained at **full** 224×224 **ImageNet resolution** on **ImageWoof** (10 fine-grained dog breed classes from ImageNet, $\approx$ 9,469 train / 3,925 val images), loaded unmodified via `timm.create_model("nf_resnet50", pretrained=False, num_classes=10)`. Because ImageWoof has $\sim$ 9.5k training images (3,700 steps per 100 epochs at `batch_size=256`), it tests unmodified 224×224 spatial feature hierarchies at roughly one-fifth the step count of CIFAR-100.

V-A3. Depthwise ConvNet (`ConvNeXt-V2-Tiny` [24], ≈ 28.6M params): Uses 7×7 depthwise convolutions, `LayerNorm`, and Global Response Normalization (`GRN`) with Mixup (0.8), CutMix (1.0), and Label Smoothing (0.1), trained on **Mini-ImageNet** (128×128, 100

ImageNet classes, 50k train / 10k val) [10], loaded via `timm.create_model("convnextv2_tiny", pretrained=False, num_classes=100)`. At 128×128, the final stage preserves a 4×4 spatial grid for the 7×7 depthwise kernels while matching CIFAR-100's exact step budget (19,500 steps at 100 epochs).

V-A4. Vision Transformer (`ViT-Small` [25], Patch 8×8, ≈ 22.0M params): Multi-head self-attention trained on **Mini-ImageNet** (128×128, 100 classes, 50k train / 10k val) following the Mini-ImageNet ViT evaluation protocol of Liang et al. [10], with Mixup (0.8), CutMix (1.0), RandAugment, and Label Smoothing (0.1), loaded via `timm.create_model("vit_small_patch16_224", pretrained=False, img_size=128, patch_size=8, num_classes=100)` ($16 \times 16 = 256$ patches/image).

V-A5. Flow-Matching Generative Model (`SiT-S/2` [27, 26], ≈ 33.0M params): Scalable Interpolant Transformer trained with Rectified Flow Matching velocity loss ($\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x_0,x_1}[\|v_\theta(x_t,t) - (x_1 - x_0)\|_2^2]$ and Model EMA 0.999) on **CIFAR-10** (32×32), loaded from `SiT_models["SiT-S/2"]`.

Epoch Range, Deterministic Shuffling, and Validation Schedule: Track V-A allows 50–150 epochs for CNNs (`V-A1`, `V-A2`) and 50–200 epochs for ConvNeXt, ViT, and SiT (`V-A3`–`V-A5`), with a **default target of 100 epochs** (19,500 steps for `V-A1/A3/A4/A5`; 3,700 steps for `V-A2`) at `batch_size = 256`. To prevent class-ordering bias in `BatchNorm2d` statistics and `Mixup/CutMix` pairings, training loaders shuffle sample indices globally at every epoch using a deterministic generator seeded with `seed + epoch` (`shuffle=True`, `drop_last=True`, `pin_memory=True`, and asynchronous `non_blocking=True` device transfers), guaranteeing that all six optimizers observe the exact same shuffled mini-batch sequence (x_t, y_t) at every step t. All six optimizers run for the exact same number of epochs E using linear warmup (5% of E for `V-A1/A2/A5`; 10% of E for `V-A3/A4`) followed by cosine decay to $\eta_{\min} = 0.05\eta$. Validation runs every 10% of total epochs and at the final epoch (`shuffle=False`, `drop_last=False`).

Table 3: Track V-A (Core): Vision From-Scratch Evaluation Matrix (`pretrained=False`, `bfloat16`)

Code	Model Backbone	Architecture	Dataset & Res.	Batch	Epochs (Range / Default)	Warmup & Schedule	Learning Rate (η)	Primary Metrics
V-A1	`ResNet-50` [23] (CIFAR 3×3 stem, ~ 23.7M)	Residual CNN (`BatchNorm2d` + `ReLU`)	CIFAR-100 (100 cls, 32×32)	256	50–150 Ep **100 Ep** (19.5k steps)	5% (5 Ep) Warmup + Cosine ($\eta_{\min} = 0.05\eta$)	LuminaV: 8e-4, 3e-4 AdamW: 1e-3 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A2	`NF-ResNet-50` [4] (Norm-Free, ~ 25.6M)	Normalization-Free CNN (Scaled Weight Std, No BN)	ImageWoof (10 cls, 224×224)	256	50–150 Ep **100 Ep** (3.7k steps)	5% (5 Ep) Warmup + Cosine ($\eta_{\min} = 0.05\eta$)	LuminaV: 8e-4, 3e-4 AdamW: 1e-3 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A3	`ConvNeXt-V2-Tiny` [24] (~ 28.6M)	Depthwise ConvNet (DW 7×7 + `LN` + `GRN`)	Mini-ImageNet (100 cls, 128×128)	256	50–200 Ep **100 Ep** (19.5k steps)	10% (10 Ep) Warmup + Cosine (Mixup/CutMix)	LuminaV: 8e-4, 3e-4 AdamW: 8e-4 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A4	`ViT-Small` [25, 10] (Patch 8×8, ~ 22.0M)	Vision Transformer (Multi-Head Self-Attention)	Mini-ImageNet (100 cls, 128×128)	256	50–200 Ep **100 Ep** (19.5k steps)	10% (10 Ep) Warmup + Cosine (RandAug/Mixup)	LuminaV: 8e-4, 3e-4 AdamW: 8e-4 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A5	`SiT-S/2` [27] (Interpolant Transformer, ~ 33.0M)	Generative Vision (Rectified Flow Matching [26])	CIFAR-10 (10 cls, 32×32)	256	50–200 Ep **100 Ep** (19.5k steps)	5% (5 Ep) Warmup + Cosine (EMA 0.999)	LuminaV: 5e-4, 3e-4 AdamW: 5e-4 | Lion: 1e-4	Held-Out Velocity MSE Loss & FID

6.4 Track V-B: Downstream Fine-Tuning Evaluation (Optional Tier)

Track V-B is an optional benchmark run when additional compute budget is available. Fine-tuning starts from pretrained weights and uses small learning rates ($\eta \leq 10^{-4}$). At these step sizes,

plain `bfloat16` updates without stochastic rounding can round to zero when $|\eta\delta_t| < \frac{1}{2}\mathrm{ULP}(\theta_t)$ (BF16 rounding underflow), while poorly scaled updates can degrade pretrained representations (Table 4):

V-B1. Object Detection Fine-Tuning (`RT-DETR-R18` / `YOLOv11-S` / `FCOS-ResNet50`):

- *Initialization & Dataset:* Pretrained COCO/ImageNet backbone with a randomly initialized detection head on `cppe-5` (5 classes, 480×480) or `Pascal VOC 2007` (20 classes, 480×480).
- *Schedule & Learning Rates:* 15–35 epochs (**default 25 epochs**), `batch_size = 16` (or 32), 2-epoch linear warmup + cosine decay. Learning rates: LuminaV $\eta \in \{1 \times 10^{-4}, 2 \times 10^{-4}\}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Loss & Metrics:* Hungarian bipartite matching / multi-task box loss (Focal + L_1 + GIoU), evaluated via **COCO mAP**$_{50:95}$, **mAP**$_{50}$, and Validation Loss every 5 epochs.

V-B2. Fine-Grained Classification Fine-Tuning (`ViT-Base/16` & `ConvNeXt-Base`):

- *Initialization & Dataset:* Pretrained ImageNet-1K backbone (`timm`) with a new linear classification head on `Oxford-IIIT Pet` (37 classes, ≈ 3,680 train / 3,669 test images at 224×224) [3].
- *Schedule & Learning Rates:* 10–30 epochs (**default 20 epochs**), `batch_size = 64`, 2-epoch linear warmup + cosine decay. Learning rates: LuminaV $\eta = 1 \times 10^{-4}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Metrics:* **Top-1 Accuracy (%)**, **Top-5 Accuracy (%)**, and Validation Cross-Entropy Loss evaluated every 2 epochs.

Table 4: Track V-B (Optional Tier): Vision Downstream Fine-Tuning Evaluation Matrix

Code	Pretrained Backbone	Downstream Task	Dataset & Res.	Batch	Epochs (Range / Default)	Warmup & Schedule	Learning Rate (η)	Primary Metrics
V-B1	`RT-DETR-R18` / `YOLOv11-S` / `FCOS-R50`	End-to-End Object Detection (Hungarian / Focal + L_1 + GIoU)	`cppe-5` (5 cls) or `VOC 2007` (480×480)	16	15–35 Ep **25 Ep** (Eval / 5 Ep)	2 Ep Linear Warmup + Cosine Decay	LuminaV: 1e-4, 2e-4 AdamW: 1e-4 | Lion: 3e-5	COCO mAP$_{50:95}$, mAP$_{50}$, Val Loss
V-B2	`ViT-Base/16` [25] & `ConvNeXt-Base` [24]	Fine-Grained High-Res Transfer Classification	`Oxford-IIIT Pet` (37 cls, 224×224)	64	10–30 Ep **20 Ep** (Eval / 2 Ep)	2 Ep Linear Warmup + Cosine Decay	LuminaV: 1e-4 AdamW: 1e-4 | Lion: 3e-5	Top-1 & Top-5 Acc, Val CE Loss

6.5 Acceptance Criteria for Vision (V-Eval)

Following the Two-Class structure of Stage IV, V-Eval compares candidate optimizers against the two-buffer baselines (`AdamW_Native_BF16` and `AdamW_FP32_Master`) and the single-buffer baselines (`Lion_Native_BF16` and `Lion_FP32_Master`):

Class I. Dual-Buffer / Full-State Optimizers (e.g., `LuminaV_Buffer2_Dual`): The candidate passes Class I if it satisfies the stability checks ($\mathrm{F}_1 \wedge \mathrm{F}_2$, keeping $R_{t,l} \in [10^{-7}, 0.10]$ without unrecovered loss spikes), uses no additional optimizer memory compared to `AdamW_Native_BF16` ($\Delta\mathrm{VRAM}_{\mathrm{opt}} \leq 0.0$ GB), and matches or improves validation quality on the target task:

$$\Delta\mathrm{Acc}_{\mathrm{Top\text{-}1}} \geq 0.0\%, \qquad \Delta\mathrm{mAP}_{50:95} \geq 0.0\%, \qquad \Delta\mathcal{L}_{\mathrm{FM}} \leq 0.0. \tag{13}$$

Class II. Single-Buffer / Compressed-State Optimizers (e.g., `LuminaV_Buffer1_TDCLM`): The candidate passes Class II if it satisfies the stability checks ($F_1 \wedge F_2$), reduces isolated optimizer memory by at least **35.0**% compared to `AdamW_Native_BF16` and at least **70.0**% compared to `AdamW_FP32_Master` while keeping memory within +10.0% of single-buffer `Lion_Native_BF16`:

$$\begin{aligned} \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. AdamW-BF16})} &\leq -35.0\%, \qquad \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. AdamW-FP32})} \leq -70.0\%, \\ \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. Lion-BF16})} &\leq +10.0\%, \end{aligned} \tag{14}$$

keeps per-step optimizer latency within 1.15× of `AdamW_Native_BF16` (and faster than `AdamW_FP32_Master`), keeps final validation loss within $\Delta\mathcal{L}_{\text{val}} \leq 0.25$ nats (or Top-1 accuracy within −1.5%) of two-buffer `AdamW_Native_BF16`, and achieves equal or better validation quality than single-buffer `Lion_Native_BF16` ($\Delta\text{Acc}_{\text{Top-1}}^{(\text{vs. Lion-BF16})} \geq 0.0\%$ or $\Delta\mathcal{L}_{\text{val}}^{(\text{vs. Lion-BF16})} \leq 0.05$ nats).

7 Differential Versioning & Regression Protocol (D-Eval)

When updating an existing optimizer or backend kernel from version v_{k-1} to v_k, LSO-Eval uses the **D-Eval Protocol** to check that code changes do not introduce numerical or performance regressions.

7.1 The Four Regression Gates

A release candidate is checked across four gates before release (Table 5):

Table 5: The D-Eval Four-Gate Regression Checklist

Gate	Evaluation Target	Acceptance Criterion	Verification Tool
Gate 1	Algebraic Safety	All Stage I SMT queries return `UNSAT`; approximation and SR bounds preserved.	Z3 [1] / SymPy [12]
Gate 2	Trajectory & Backend Parity	Head-to-head 2D run on $\kappa \geq 1000$ and cross-backend parity tests pass.	2D Sim & Backend Tests
Gate 3	Checkpoint Compatibility	Checkpoints saved in version v_{k-1} load cleanly into v_k with identical dtypes and trajectories.	State-Dict Audit
Gate 4	Empirical Metrics Delta	Meets non-regression bounds: $\Delta\mathcal{L} \leq 0$, $\Delta\text{VRAM}_{\text{opt}} \leq 0$, $\Delta T_{\text{kernel}} \leq 0$ (optional when update trajectory is invariant).	Stage IV (Optional)

7.2 Performance Delta Comparison

For Gate 4, the team compares the new version against the previous version:

$$\Delta\mathcal{L}_{\text{EMA}} \leq 0, \qquad \Delta\text{VRAM}_{\text{opt}} \leq 0, \qquad \Delta T_{\text{kernel}} \leq 0, \tag{15}$$

where $\Delta X = X(v_k) - X(v_{k-1})$. If any metric regresses ($\Delta > 0$), the team documents the reason and trade-off in the project changelog.

Reference Case Example: D-Eval Regression Summary Across the LuminaV Release Series

Summary of D-Eval regression checks across the LuminaV release series:

- **Gate 1 (SMT & Unified ϵ_{eff}):** Unified Convex Lookahead Momentum weights ($w_1 = \beta_1^2, w_2 = 1 - \beta_1^2$) across Buffer 1 and Buffer 2 ($w_1 + w_2 = 1$), verified [13/6] Padé tanh accuracy ($9.48 \times 10^{-8} < 1.5 \times 10^{-6}$), and promoted pure `FP16` 16-bit state containers to in-container Bitwise-BF16 (8-bit exponent range 1.18×10^{-38} at no extra VRAM), unifying $\epsilon_{\text{eff}} = \epsilon = 10^{-8}$ across all precisions.
- **Gate 2 (Uncentered Defect Simplification & Backend Parity):** Standardized on coordinate-wise gradient-momentum residuals $d_t^{(\text{B1})} = g_t - m_{t-1}$ and $d_t^{(\text{B2})} = g_t - m_t$, removing pre-step mean reduction kernels while preserving trajectory parity with `v1.5.3` (`centered_defect=False`) across Scalar, AVX2, AVX-512, Vulkan 1.3/1.4 Wave32 UMA, and Triton AOT/JIT.
- **Gate 3 (Contiguous Bucket Fusion & Checkpoint Cleanliness):** Added contiguous parameter coalescing (`fuse_tensors=False`, `bucket_cap_mb=500.0`) and Triton JIT pre-warmup (`prewarm_jit=True`), resetting 6-slot `float32` scratchpad accumulators (s_0, s_1, s_2, s_5) to zero on export without leaking staging keys.
- **Gate 4 (Optimizer Memory Footprint):** Verified in `v1.5.3` Stage IV runs with $\Delta\text{VRAM}_{\text{opt}} = 0$ GB (7.94 GB isolated optimizer state on 4B LLM for Buffer 1; 15.86 GB for Buffer 2) and packaged standalone CPython Stable ABI wheels (`abi3-py310/abi3t`).

References

[1] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Budapest, Hungary, 2008, pp. 337–340.

[2] L. Maligranda, "Some remarks on the triangle inequality for norms," *Banach J. Math. Anal.*, vol. 2, no. 2, pp. 31–41, 2008.

[3] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 49205–49233.

[4] A. Brock, S. De, S. L. Smith, and K. Simonyan, "High-performance large-scale image recognition without normalization," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 1059–1071.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, 2019.

[6] J. Zhuang, T. Tang, Y. Ding, S. Tatikonda, N. Dvornek, X. Papademetris, and J. S. Duncan, "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 18795–18806.

[7] S. Taniguchi, K. Harada, G. Minegishi, Y. Oshima, S. C. Jeong, G. Nagahara, T. Iiyama, M. Suzuki, Yusuke Iwasawa, and Y. Matsuo, "ADOPT: Modified Adam can converge with any β_2 with the optimal rate," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[8] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Stockholm, Sweden, 2018, pp. 1842–1850.

[9] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, "GaLore: Memory-efficient LLM training by gradient low-rank projection," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, vol. 235, Vienna, Austria, 2024.

[10] K. Liang, L. Chen, B. Liu, and Q. Liu, "Cautious optimizers: Improving training with one line of code," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[11] M. R. Zhang, J. Lucas, G. E. Hinton, and J. Ba, "Lookahead optimizer: k steps forward, 1 step back," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 9593–9604.

[12] A. Meurer *et al.*, "SymPy: Symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, p. e103, Jan. 2017, doi: 10.1717/peerj-cs.103.

[13] A. Molina, P. Schramowski, and K. Kersting, "Padé activation units: End-to-end learning of flexible activation functions in deep networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[14] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 1737–1746.

[15] P. Zamirai, J. Zhang, C. R. Aberger, and C. De Sa, "Revisiting BFloat16 training," *arXiv preprint arXiv:2010.06192v2*, Mar. 2021.

[16] M. Jamil and X.-S. Yang, "A literature survey of benchmark functions for global optimization problems," *Int. J. Math. Model. Numer. Optim.*, vol. 4, no. 2, pp. 150–194, 2013, doi: 10.1504/IJMMNO.2013.055204.

[17] H. H. Rosenbrock, "An automatic method for finding the greatest or least value of a function," *Comput. J.*, vol. 3, no. 3, pp. 175–184, Oct. 1960, doi: 10.1093/comjnl/3.3.175.

[18] L. A. Rastrigin, *Systems of Extremal Control*. Moscow, Russia: Nauka, 1974.

[19] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, New Orleans, LA, USA, 2022, pp. 16344–16359.

[20] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop Mach. Learn. Program. Lang. (MAPL)*, Phoenix, AZ, USA, Jun. 2019, pp. 10–19, doi: 10.1145/3315508.3329973.

[21] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[22] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[24] S. Woo *et al.*, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Vancouver, BC, Canada, 2023, pp. 16133–16142.

[25] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[26] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, 2023.

[27] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie, "SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Milan, Italy, 2024.

Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.1)
Technical Specification — Lumina Moon (cloverx-id)